

Assignment 4

Group 2

2026-04-25

Concept and Purpose

The purpose of our Shiny app is to provide an interactive and visually engaging representation of measles cases reported in the United States since January 1, 2025. Although measles was declared eliminated in the United States in 2000 by the CDC, recent years have shown a concerning resurgence in reported cases. While current case counts remain far below the levels observed before the widespread use of the measles vaccine, the nation is experiencing the highest number of measles cases seen since the early 1990s. This trend highlights the continued public health importance of monitoring outbreaks and understanding patterns of transmission. Our app is designed to make complex surveillance data easier to interpret by transforming raw case counts into accessible maps, charts, and summaries. By allowing users to explore measles activity geographically and over time, the app aims to improve awareness of outbreak trends, support public health education, and provide a centralized tool for examining the reemergence of a vaccine-preventable disease in the United States.

Initial Plan / Motivation

Our initial plan for the Shiny app was to create a comprehensive dashboard that allowed users to analyze measles data at both the county and state levels. We intended for the app to display case counts stratified by multiple key factors, including whether infections were imported or locally acquired, as well as whether affected individuals were vaccinated or unvaccinated. Organizing the data based on these parameters would provide a robust and detailed understanding of how outbreaks are occurring across the country. To enhance usability, we planned separate tabs for county-level and state-level summary statistics so users could easily compare regions and identify areas with elevated case counts. In addition, we proposed a correlation analysis section to explore relationships between important outbreak variables. Specifically, this portion of the app was motivated by questions such as whether increases in unvaccinated cases are associated with greater local transmission, and whether higher numbers of imported cases contribute to additional local spread. These analytical features were intended to move the app beyond simple visualization and toward meaningful epidemiological insight.

Initial concepts for Shiny app layouts are shown below:

This layout has tabs and tables.

U.S. Measles Case Explorer

Filters

Explore measles cases by location and time.

Select State:

All

Select County:

All

Select Date Range:

2025-01-16 to 2026-04-02

☒ Show points on graph

OverviewTrendsComparison

Total Reported Cases3981

Affected Locations301

Date Range2025-01-16 to 2026-04-02

Filtered Data Preview

Show 10 entries

Search:

	location_name	location_id	location_type	date	outcome_type	value	state	county
1	Adams, Colorado	8001	county	2025-08-13	case_lab-confirmed	1	Colorado	Adams
2	Adams, Colorado	8001	county	2025-08-15	case_lab-confirmed	2	Colorado	Adams
3	Adams, Colorado	8001	county	2025-08-28	case_lab-confirmed	1	Colorado	Adams
4	Adams, Colorado	8001	county	2026-03-06	case_lab-confirmed	5	Colorado	Adams
5	Alachua, Florida	12001	county	2026-01-25	case_lab-confirmed	1	Florida	Alachua
6	Alachua, Florida	12001	county	2026-02-07	case_lab-confirmed	1	Florida	Alachua
7	Allegan, Michigan	26005	county	2025-06-06	case_lab-confirmed	1	Michigan	Allegan
8	Allen, Indiana	18003	county	2025-04-07	case_lab-confirmed	1	Indiana	Allen
9	Allen, Indiana	18003	county	2025-04-09	case_lab-confirmed	5	Indiana	Allen
10	Allen, Indiana	18003	county	2025-04-23	case_lab-confirmed	2	Indiana	Allen

Showing 1 to 10 of 1,058 entries

Previous

12345...

106Next

This layout uses a map and time course plots.

United States Measles Caseload Dashboard

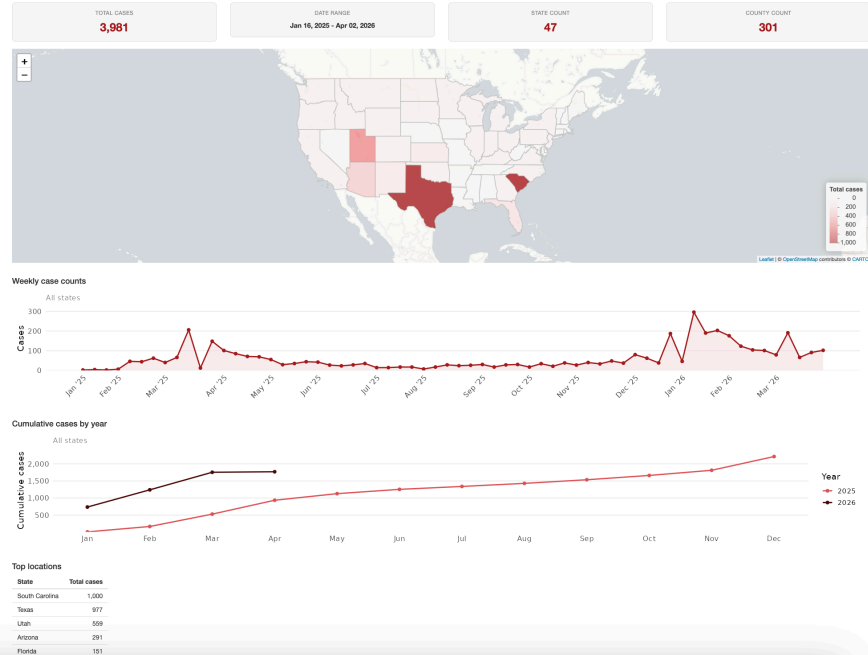
Map level
☒ State (aggregated)
☐ County

Case type
 All cases (all-confirmed)

Date range
 2025-01-16 to 2026-04-02

Filter to state(s)
 All States

Update



The final Shiny app layout is shown below:

The final layout combines features from initial concepts.

United States Measles Surveillance Dashboard

Although measles was declared eliminated in the United States in 2000 by the Centers for Disease Control and Prevention, recent years have shown a concerning resurgence in reported cases. Tracking measles cases improves awareness of outbreak trends, patterns of transmission, and potential reemergence of disease, which can aid in deploying public health initiatives to prevent further spread.

Map level
☒ State (aggregated)
☐ County

Case type
 All cases (all-confirmed)

Date range
 2025-01-16 to 2026-04-02

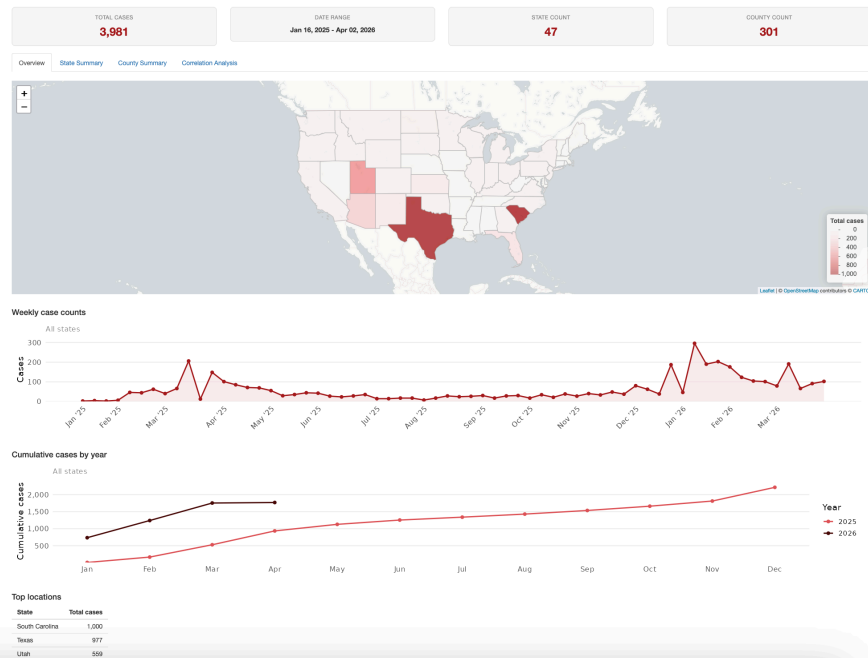
Filter to state(s)
 All States

Top number of states
 0 to 16

Top number of counties (per state)
 0 to 16

Update

Data source: CSSEGISandData from the Johns Hopkins University Measles Tracking Team - International Vaccine Access Center (IVAC), Bloomberg School of Public Health, Center for Systems Science and Engineering (CSSE), Whiting School of Engineering, Bloomberg Center for Government Excellence, Johns Hopkins University



Dataset Description

The dataset was sourced from https://github.com/CSSEGISandData/measles_data, a GitHub repository that reports data collated by the Johns Hopkins University Measles Tracking Team, which includes members from the International Vaccine Access Center, the Center for Systems Science and Engineering, and Bloomberg Center for Government Excellence. While the repository is regularly updated, our data encompasses the cases that were tracked from January 1, 2025 to April 2, 2026 from commit 2b59391dc10d791051261f5ea8ebbec234ed432. Within this repository, we selected the file `measles_county_all_updates_detailed.csv`, which includes the location of the case (in the format County, State), the Federal Information Processing Standards (FIPS) code for that location, the date of the case (in the format YYYY-MM-DD), the outcome case type (including imported, local, unvaccinated, vaccinated cases), and the value (the count of the confirmed cases that fit into all of the above named specifications). Note that some states only have data available for a subset of the case type.

App Description

For the project, an R Shiny app called "United States Measles Surveillance Dashboard" was created. The app contains five main data analysis displays, which are populated with data gated by filters on the sidebar panel. The first main display is for summary metrics, which include the total number of measles cases, the date range, number of states, and number of counties included based on the user-defined filter settings. The remaining displays appear as tabs below the summary metrics, including the Overview tab, States Summary tab, County Summary tab, and Correlations Analysis tab. The inputs, outputs, and server logic for each of the data analysis displays are described below.

Sidebar Panel

The sidebar panel, implemented using the functions `sidebarLayout` and `sidebarPanel`, contain settings that the user can specify to filter the data. The first one is the map-level selection using `radioButton` for either state or county. If state is selected, state-level geographical boundaries are shown on the map, and the measles cases count is per state. If county is selected, county-level boundaries are shown, and the measles case count is per county. The next filter setting is a single-selection dropdown menu for the measles case type using `selectInput`. Following that is a calendar with `dateRangeInput` where the user can select the start and end dates for the analyses. Below that is a multi-selection dropdown menu for states to include using `selectizeInput`. Next, two sliders with `sliderInput` are included, one for the number of top states and another for the number of top counties per state, which determine the data displayed in table summaries. Finally, an `actionButton` called "Update" is used to apply the filter settings and update the data displays.

Filter type	Input variable name
Map-level selection	<code>input\$map_level</code>
Measles case type	<code>input\$outcome</code>
Date range	<code>input\$date_range</code>
State selection	<code>input\$state_filter</code>
Slider	<code>input\$top_n_states</code>
Slider	<code>input\$top_n_counties</code>
"Update" button	<code>input\$update</code>

Main Panel

The main panel is created with the function `mainPanel` and contains all of the data display outputs, including the summary metrics with `fluidRow` and each of the tabs with `tabsetPanel` and `tabPanel`.

Reactives

The reactives in the analyses are defined here to provide context for the in-depth descriptions of each tab in the subsequent sections. Three types of reactives are used in the analyses: helper function reactives, data filter reactives, and data reactives. The helper function reactives are general support functions whose inputs are dynamically updated variables based on the filter settings. The data filter reactives are used as a preliminary filter on the raw data using the filter settings. The data reactives apply additional filters and data wrangling required for a specific output or data display. The table below outlines all the reactives defined in the R script. In the table, note that `Input` may include input variables from the UI or other reactives.

Reactive	Type	Input	Description	Usage
<code>sel_states</code>	helper	<code>input\$state_filter</code>	returns vector of selected state names with option <code>All States</code> removed, otherwise returns <code>All States</code>	any situation where list of states is needed to filter raw data
<code>plot_subtitle</code>	helper	<code>sel_states</code>	builds subtitle string for overview figures. returns <code>All states</code> if no states selected; returns names if 1-3 states selected; returns count of states if > 3 states selected	Overview tab - weekly case count figure, cumulative cases figure
<code>filtered_raw</code>	data filter	<code>input\$date_range, input\$outcome, sel_states, input\$update</code> . uses helper function <code>filter_by_date_state</code>	applies the date range, outcome type, and state filters to raw dataset, and returns a filtered data frame; gated to <code>input\$update</code> via <code>bindEvent</code>	serves as the preliminary data filter for all downstream summary reactives

Reactive	Type	Input	Description	Usage
<code>filtered_state</code>	data filter	<code>filtered_raw, input\$update</code>	group filtered data by state; gated to <code>input\$update</code> via <code>bindEvent</code>	state-level map, the state top locations table, and the state count summary box
<code>filtered_county</code>	data filter	<code>filtered_raw, input\$update</code>	group filtered data by county; gated to <code>input\$update</code> via <code>bindEvent</code>	county-level map, the county top locations table, and the county count summary box
<code>weekly_cases</code>	data	<code>filtered_raw(), input\$update</code>	group by week	Overview tab - weekly case counts figure
<code>cumulative_cases</code>	data	<code>filtered_raw(), input\$update</code>	group by year and month	Overview tab - cumulative cases by year figure
<code>state_geo</code>	data	<code>sel_states, filtered_state()</code>	downloads state boundary shapefiles and is joined count of selected states	Overview tab - Map
<code>county_geo</code>	data	<code>sel_states(), filtered_county()</code>	downloads county boundary shapefiles and is joined to county count of selected states	Overview tab - Map
<code>state_summary_data</code>	data	<code>filtered_raw(), input\$date_range, sel_states(), input\$top_n_states, input\$update</code>	gated to <code>input\$update</code> via <code>bindEvent</code>	County Summary tab
<code>county_summary_data</code>	data	<code>filtered_raw(), input\$date_range, sel_states(), input\$top_n_states, input\$top_n_counties, input\$update</code>	gated to <code>input\$update</code> via <code>bindEvent</code>	County Summary tab
<code>corr_data</code>		<code>input\$date_range, sel_states(), input\$update</code> . uses helper function <code>filter_by_date_state</code> and <code>ensure_outcome_cols</code>	gated to <code>input\$update</code> via <code>bindEvent</code>	Correlation Analysis tab

Summary Metrics

The summary metrics panel consists of informational boxes with the following information: total number of measles cases, the date range, number of states, and number of counties. This information is updated based on the user-defined filter settings.

The table below summarizes the input variables or reactives, output variables, and functions used on the sever to render the outputs.

Display	Type	Sidebar Filters	Input	Output	Server
Total cases	box	case type, date range, states, update button	sum of <code>filtered_raw()\$value</code> , where <code>value</code> is case count	<code>output\$box_total</code>	<code>renderText</code>
Date range	box	date range, update button	<code>input\$date_range, input\$update</code>	<code>output\$box_dates</code>	<code>renderText, bindUpdate</code>
State count	box	case type, date range, states, update button	<code>n_distinct(filtered_state()\$state)</code>	<code>output\$box_states</code>	<code>renderText</code>
County count	box	case type, date range, states, update button	<code>filtered_county</code> filtered to the number of counties with nonzero cases	<code>output\$box_counties</code>	<code>renderText</code>

Overview

The Overview tab contains data displays that allows the user to explore the dataset at a high level to get a sense of the measles caseload burden geographically and across time. The interactive map summarizes the total case counts for each state or county. The color scale reflects the number of case counts, where the shade of red darkens based on the case count value. In the filter settings, when the map level `state` is selected, the state boundaries are shown with total case counts computed per state. When the map level `county` is selected, the county boundaries are shown with total case counts computed per county. (Note that toggling between `state` or `county` for the map level automatically updates the map boundaries without

using the "Update" button so the user can view both displays with ease. This toggle does not impact the data filtering or wrangling on the backend for other data views.) When users hover over the state or county, the label is shown (either **state** or **county, state**) along with total case count. If specific states are selected in the filter settings, the case count color coding is applied, and all other non-selected states are grayed out. Only data from the selected case type and date range are shown.

Note that a known bug is that there could be minor mismatches between the the county label used in the dataset and the **tigris** package. For example, the dataset lists Upstate as a county in South Carolina. However, Upstate is a descriptor for multiple counties in northern South Carolina. As such, the **tigris** package codes the counties separately, as there is no actual county named Upstate in South Carolina. The result is that case counts for the individual counties are 0, which is technically inaccurate.

Below the map are two figures that summarize measles case loads over time. The data included in these figures is based on the filter settings, specifically the states, case type, and date range. The figure subtitles display information about the states selected. The full state names are listed for up to three states. If more than three states are selected, the number of states is shown. Alternatively, the text **All states** is displayed if that is the option selected.

The first figure below the map is the weekly case counts, with weeks on the x-axis and total cases on the y-axis. Note that the x-axis labels are per month for ease of display. This figure allows the user to visualize when cases begin to increase over time and identify specific weeks when case loads peak. The second figure is the cumulative cases by year, with months on the x-axis and cumulative cases on the y-axis. Years are color coded according to the legend. With this figure, the user can compare caseload trends between different years, in particular for understanding whether the caseload in the current year is improving or worsening compared to previous years. In a real-world situation, the effectiveness of public health initiatives such as vaccination clinics can be generally assessed with visualizations like these to see if caseloads drop over time in specific locations.

The top locations table lists the locations with the most case counts in descending order. When the map level **state** is selected, top states are shown in the table. When the map level **county** is selected, top counties are shown in the table. The **top number of states** slider in the filter settings determines the number of rows in the table.

The table below summarizes the input variables or reactives, output variables, and functions used on the sever to render the outputs.

Display	Type	Sidebar Filters	Input	Output	Server	Helper functions
Map	figure	map level, case type, date range, states, update button	<code>input\$map_level</code> , <code>state_geo()</code> or <code>county_geo()</code> , <code>input\$update</code>	<code>output\$map</code>	<code>renderLeaflet</code> , <code>observe</code> block, <code>bindEvent</code>	<code>make_pal</code>
Weekly case counts	figure	case type, date range, states, update button	<code>weekly_cases</code> , <code>plot_subtitle()</code>	<code>output\$line_plot</code>	<code>renderPlot</code>	<code>plot_subtitle</code>
Cumulative cases by year	figure	case type, date range, states, update button	<code>cumulative_cases</code> , <code>plot_subtitle()</code>	<code>output\$cumulative_plot</code>	<code>renderPlot</code>	<code>plot_subtitle</code> , <code>year_colors</code>
Top locations	table	map level, case type, date range, states, top states slider, update button	<code>input\$map_level</code> , <code>input\$top_n_states</code> , <code>filtered_state()</code> or <code>filtered_county()</code>	<code>output\$top_table</code>	<code>renderTable</code>	

States Summary

The States Summary tab contains an interactive table of summary statistics of the data selected in the analysis at the state level. Data can be sorted in ascending or descending order by clicking on the column title. The data shown is based on the filter settings for case type, date range, and selected states. The **top number of states** slider in the filter settings determines the number of rows in the table, with a maximum of 10.

The columns in the table include the following: state, total count for cases, range, mean, standard deviation, median, interquartile range, proportion of imported cases, proportion of local cases, incidence rate ratio, and number of affected counties.

The descriptive statistics columns (total count, range, mean, standard deviation, median, interquartile range, number of affected counties) are based on the case type selected. The statistics are calculated over counties within a given state.

The proportion and rate columns (proportion of imported cases, proportion of local cases, incidence rate ratio) are based on all case types. The proportion of imported cases is calculated as the number of imported cases divided by the sum of imported and local cases. The proportion of local cases is calculated as the number of imported cases divided by the sum of imported and local cases. The incidence rate ratio (IRR) is calculated as the ratio of unvaccinated cases to vaccinated cases.

- IRR > 1: indicates that measles incidence is higher in the unvaccinated group than in the vaccinated group; as such, the vaccine may reduce the risk of disease
- IRR = 1: indicates that measles incidence is the same in both vaccinated and unvaccinated groups; as such, this may be an initial warning of waning vaccine immunity
- IRR < 1: indicates that measles incidence is lower in the unvaccinated group than in the vaccinated group; as such, this may be due to severe waning vaccine immunity

The table below summarizes the input variables or reactives, output variables, and functions used on the sever to render the outputs.

Display	Type	Sidebar Filters	Input	Output	Server	Helper functions
State summary	table	case type, date range, states, top states slider, update button	<code>state_summary_data()</code>	<code>output\$state_summary_table</code>	<code>renderDataTable</code>	<code>build_outcome_wide (filter_by_date_state, ensure_outcome_col, add_proportion_cols)</code>

County Summary

The County Summary tab contains an interactive table of summary statistics of the data selected in the analysis at the county level. Data can be sorted in ascending or descending order by clicking on the column title. The data shown is based on the filter settings for case type, date range, and selected states. Both the `top number of states` slider and `top number of counties (per state)` slider in the filter settings determine the number of rows in the table. For example, with the default settings, `top number of states` is 5 and `top number of counties (per state)` is 3. This means that for the top 5 states, data for the top 3 counties in each of those states is shown. The same columns are used as in the State Summary, with the addition of a column for county. Note that at the county level, the statistics and other metrics are calculated over reporting days for a given county.

The table below summarizes the input variables or reactives, output variables, and functions used on the sever to render the outputs.

Display	Type	Sidebar Filters	Input	Output	Server	Helper functions
County summary	table	case type, date range, states, top states slider, top counties slider, update button	<code>county_summary_data()</code>	<code>output\$county_summary_table</code>	<code>renderDataTable</code>	<code>build_outcome_wide (filter_by_date_state, ensure_outcome_col, add_proportion_cols)</code>

Correlation Analysis

The Correlation Analysis tab is for performing exploratory correlations between predefined case types in the dataset to address two scientific questions. The data filters applied are the date range and selected states. Note that case types are fixed by the correlation being examined, thus the case type filter is not used.

The first inquiry is understanding the relationship between imported cases and local transmission. Imported cases are known cases where measles was contracted outside of the given location. Whether increasing imported cases lead to higher local transmission is investigated. The second inquiry is understanding the relationship between unvaccinated cases and local transmission. Whether increasing unvaccinated individuals with measles lead to higher local transmission is examined. If this information is tracked, public health initiatives can be deployed to locations with high imported or unvaccinated cases to block the spread of local transmission.

Figure 1 is a scatter plot for imported cases versus local transmission cases. Figure 2 is a scatter plot for unvaccinated cases versus local transmission cases. In both figures, a linear trend line is shown with shading as the 95% confidence interval. The Spearman correlation was performed, since it does not require the assumption of normality and is less sensitive to outliers. The statistics summary tables below each figure list the correlation value (ρ), p-value, and sample size. The analyses uses data at the county level based on the selected states.

The table below summarizes the input variables or reactives, output variables, and functions used on the sever to render the outputs.

Display	Type	Sidebar Filters	Input	Output	Server	Helper functions
Fig 1 (imported vs local cases)	figure	date range, states, update button	<code>corr_data()</code>	<code>output\$corr_plot_imp_local</code>	<code>renderPlot</code>	<code>build_corr_plot</code>
Stats summary for Fig 1	table	date range, states, update button	<code>corr_data()</code>	<code>output\$corr_table_imp_local</code>	<code>renderTable</code>	<code>build_corr_table</code>
Fig 2 (unvaccinated vs local cases)	figure	date range, states, update button	<code>corr_data()</code>	<code>output\$corr_plot_unvacc_local</code>	<code>renderPlot</code>	<code>build_corr_plot</code>
Stats summary for Fig 2	table	date range, states, update button	<code>corr_data()</code>	<code>output\$corr_table_unvacc_local</code>	<code>renderTable</code>	<code>build_corr_table</code>

Analyses and Insights

For this sample analyses, the default filter settings are used. This includes the case type as **All cases (lab-confirmed)**, all dates (**2025-01-16 to 2026-04-02**), all states, maximum of 5 for **top number of states**, and maximum of 3 for **top number of counties (per state)**.

On the summary metrics upper panel, we see a total number of 3,981 cases from 47 states and 301 counties within the United States. The Overview tab describes the geographical trends and time course of the measles caseload. At the state level, it is evident that Texas and South Carolina visually stand out as states with the highest number of cases. At the county level, Gaines, Texas, and Mohave, Arizona are the counties with the highest number of cases. (Note the known data display bug as described in the Overview section above, regarding data labeling issues that may impact the display of county-level data on the map.) In the weekly case counts figure, the caseload peaks are sometime during March 2025 with more than 200 cases and January 2026 with more than 300 cases. Generally, caseloads are higher than baseline from February-May 2025 to January-April 2026. In the cumulative cases by year figure, the measles caseload is higher overall as compared to last year within the January to April timeframe. The top locations table confirms what was observed on the map views. South Carolina, Texas, Utah, Arizona, and Florida have the highest cases nationally. In addition, these counties have the highest cases: Upstate, South Carolina, Gaines, Texas, Mohave, Arizona, Southwest Health District, Utah, and Hudspeth, Texas.

In the State Summary tab, the top 5 states are listed in a table in descending order of case count. A few interesting observations can be made. South Carolina has a very high incidence rate ratio (IRR), meaning that unvaccinated cases are higher than vaccinated cases. That certainly aligns with the prior observation that South Carolina is one of the states with the highest number of cases, and contains a county with one of the highest caseloads. In addition, Texas has the highest number of affected counties, which may speak to greater within-state disease spread. Another noteworthy metric is that Florida has the highest proportion of imported cases.

In the County Summary tab, the top 3 counties from each of the top 5 states are listed in a table in descending order of case count. Similar observations can be made as the State Summary in terms of IRR. With the county-level data, it is clear that the high cases in Upstate, South Carolina and Southwest Health District, UT contribute to that. Another observation is that Hudspeth, Texas has the highest mean and median cases, meaning it has the most cases reported per day.

The Correlation Analysis tab shows results from Spearman rank correlations that were performed. Spearman correlations are used since it does not require the assumption of normality and is less sensitive to outliers. All available county-level data for the specified case types are included. For the imported cases versus local transmission correlation, a negative correlation (-0.366) with $p < 0.001$ was found. One interpretation is that herd immunity is effective at blocking the spread of measles, since imported cases enter a location whose population has a high vaccination coverage, leaving fewer susceptible people to infect. For the unvaccinated cases versus local transmission correlation, a weak positive correlation (0.126) with $p = 0.029$ was found. This may suggest that gaps in vaccination may be driving the local spread of measles. One of the challenges with this analyses is that data for a given case type are not available for some of the states and/or counties, which may skew analyses at the national level.

GitHub and Shiny App Deployment

The code for the Shiny app can be found on the team's public GitHub repository [here](#).

The app is hosted online at shinyapps.io [here](#).

References

U.S. Measles Cases Hit Highest Level Since Declared Eliminated in 2000 | International Vaccine Access Center. (2025, July 7). International Vaccine Access Center. <https://publichealth.jhu.edu/ivac/2025/us-measles-cases-hit-highest-level-since-declared-eliminated-in-2000>

Centers for Disease Control and Prevention. (2025, April 25). Measles cases and outbreaks. Measles (Rubeola); CDC. <https://www.cdc.gov/measles/data-research/index.html>

Centers for Disease Control and Prevention. (2024, May 9). History of Measles. Centers for Disease Control and Prevention. <https://www.cdc.gov/measles/about/history.html>

Measles cases in the United States. (2025). Our World in Data. <https://ourworldindata.org/grapher/number-of-measles-cases>

Shiny App Code

Description

```
#####
##### Install and load packages
#####

# The lines below are run if the packages were not yet installed.
# Otherwise, they are commented out so the code is not executed.
# install.packages(c(
#   "shiny", "leaflet", "tigris", "sf", "scales", "lubridate", "tidyverse", "DT"
# ))

# The shiny package is used to build the interactive web application.
library(shiny)
# The leaflet package is used to render the interactive choropleth map.
library(leaflet)
# The tigris package is used to download US Census shapefiles for state
# and county boundaries.
library(tigris)
# The sf package is used to handle spatial data formats returned by tigris.
library(sf)
# The scales package contains the function comma, which formats large
# numbers with comma separators for readability (e.g., 1234 becomes 1,234).
library(scales)
# The lubridate package contains date functions including floor_date, year,
# and month, which are used to round dates and extract date components.
library(lubridate)
# The tidyverse package contains functions for data wrangling and
# generating figures, such as `ggplot`, among others.
library(tidyverse)
# The DT package provides the dataTableOutput and renderDataTable functions
# used to display the interactive summary tables.
library(DT)

# The tigris option below instructs the package to save downloaded shapefiles
# to a local cache folder. This prevents re-downloading on each app run.
options(tigris_use_cache = TRUE)

#####
##### Data loading and preparation
#####

# File paths are handled differently in developer mode vs when app is deployed.
# If users are testing changes to the app locally, developer mode is set to TRUE.
# Otherwise, if the app is deployed, developer is set to FALSE.
developer_mode <- FALSE

if (developer_mode) {
  wd_path <- getwd()
  data_path <- paste0(wd_path, "/Assignment-4/measles_county_all_updates_detailed.csv")
} else {
  data_path <- "measles_county_all_updates_detailed.csv"
}
```

```

# The data file is read into the R environment using read_csv from the readr
# package. An object called raw is created which contains the imported data.
raw <- read_csv(data_path, header=TRUE) %>%
  mutate(
    # The date column is converted from a character string to an R Date object
    # so that date-based filtering and arithmetic can be performed.
    date = as.Date(date),

    # The location_id column is zero-padded to 5 digits to match the format
    # of US Census FIPS codes used by the tigris shapefiles.
    # For example, 8001 becomes "08001" for Adams County, Colorado.
    fips = sprintf("%05d", as.integer(location_id)),

    # The state name is extracted from the location_name column, which is
    # formatted as "County, State". The sub function removes everything up to
    # and including the comma and space. trimws removes any remaining whitespace.
    state = trimws(sub(".*,\\s*", "", location_name)),

    # The county name is extracted from the location_name column by removing
    # everything from the comma onward.
    county = trimws(sub(",.*", "", location_name)),

    # The value column is explicitly converted to numeric to ensure case counts
    # are stored as numbers and not character strings.
    value = as.numeric(value),

    # Each date is rounded down to the start of its week using floor_date.
    # For example, a Wednesday date becomes the preceding Sunday.
    # This column is used for grouping in the weekly case count plot.
    week = floor_date(date, "week")
  )

# A named character vector of outcome types is created for use in the
# case type dropdown input. The left side of each pair is the label shown
# to the user, and the right side is the value stored in the data column.
outcome_choices <- c(
  "All cases (lab-confirmed)" = "case_lab-confirmed",
  "Imported cases"           = "case_imported",
  "Local transmission"       = "case_local",
  "Unvaccinated"              = "case_unvaccinated",
  "Vaccinated"                = "case_vaccinated"
)

# The earliest and latest dates in the dataset are stored as a two-element
# vector. These values are used to set the minimum, maximum, and default
# values of the date range input in the user interface.
date_range <- range(raw$date, na.rm = TRUE)

# The unique calendar years present in the dataset are extracted and sorted.
# This vector is used to build a color palette for the cumulative plot
# that scales automatically if the data spans more than two years.
data_years <- sort(unique(year(raw$date)))

```

```
#####
##### Plain (non-reactive) helper functions
#####

### Color palette

# A color palette function for the map is defined. It accepts a vector of
# case count values and returns a leaflet-compatible color scale ranging
# from light gray at zero cases to dark red at the maximum case count.
# The na.color argument specifies the color for counties or states with
# no matching data after the join.
make_pal <- function(values) {
  colorNumeric(
    palette = colorRampPalette(c("#F5F5F5", "#FADADD", "#F4A0A0",
                                "#E05555", "#A61C1C"))(256),
    domain = c(0, max(values, 1)),
    na.color = "#cccccc"
  )
}

# A named color vector for the cumulative plot is created using
# colorRampPalette, which interpolates between two red shades to produce
# one color per year. The setNames function pairs each color with its
# corresponding year string so that scale_color_manual in ggplot2 can
# match colors to the correct year lines.
year_colors <- setNames(
  colorRampPalette(c("#E05555", "#4A0A0A"))(length(data_years)),
  as.character(data_years)
)

### Data filter

# This function applies the date range and optional state filter to any
# data frame that has "date" and "state" columns. It is called inside the
# reactives below.
# Inputs include:
# df - the data frame to filter (usually raw or a subset of it)
# date_start - earliest date to keep (from input$date_range[1])
# date_end - latest date to keep (from input$date_range[2])
# states - character vector of selected state names, or an empty
#          vector (length 0) meaning "keep all states"
filter_by_date_state <- function(df, date_start, date_end, states) {
  # Step 1: always apply the date range filter
  out <- df %>%
    filter(date >= date_start, date <= date_end)
  # Step 2: only apply the state filter when specific states were chosen.
  # If states is empty (length 0), every state is retained.
  if (length(states) > 0) {
    out <- out %>% filter(state %in% states)
  }
  out
}
```

```

### Table setup

# This function guarantees a set of required column names exist in a data
# frame. It is called after pivot_wider in any block that needs case_imported,
# case_local, etc., because pivot_wider only creates columns that are present
# in the data. If a filtered subset happens to have no rows for a given outcome
# type, that outcome column would be missing entirely - adding it as a zero
# column prevents downstream division errors.
# Inputs include:
# df - the data frame to check and modify
# cols - a character vector of column names that must exist
# It returns the data frame with any missing columns added as zero.
ensure_outcome_cols <- function(df, cols) {
  for (col in cols) {
    if (!col %in% names(df)) df[[col]] <- 0
  }
  df
}

# This function adds Prop Imported, Prop Local, and IRR columns to any
# data frame that already has case_imported, case_local, case_unvaccinated,
# and case_vaccinated columns (i.e., after pivot_wider). NaN and Inf are
# replaced with NA so the table displays a dash instead of a number.
add_proportion_cols <- function(df) {
  df %>%
    mutate(
      `Prop Imported` = round(case_imported / (case_imported + case_local), 3),
      `Prop Imported` = ifelse(is.nan(`Prop Imported`), NA, `Prop Imported`),
      `Prop Local`    = round(case_local / (case_imported + case_local), 3),
      `Prop Local`    = ifelse(is.nan(`Prop Local`), NA, `Prop Local`),
      `IRR`           = round(case_unvaccinated / case_vaccinated, 3),
      `IRR`           = ifelse(is.nan(`IRR`) | is.infinite(`IRR`), NA, `IRR`)
    )
}

# This function builds the Part B proportion/rate data frame used by both
# summary reactives. It filters raw to the selected date range and states,
# then pivots outcome types wide and computes proportion and IRR columns.
# group_cols - character vector of grouping columns before outcome_type,
#               e.g. "state" for state-level or c("state", "county") for
#               county-level. The same vector is used in the final select
#               so the returned data frame has exactly those ID columns
#               plus Prop Imported, Prop Local, and IRR.
build_outcome_wide <- function(raw, date_start, date_end, states, group_cols) {
  filter_by_date_state(raw, date_start, date_end, states) %>%
    group_by(across(all_of(c(group_cols, "outcome_type")))) %>%
    summarize(total = sum(value, na.rm = TRUE), .groups = "drop") %>%
    pivot_wider(names_from = outcome_type, values_from = total,
                values_fill = 0) %>%
    ensure_outcome_cols(c("case_imported", "case_local",
                          "case_unvaccinated", "case_vaccinated")) %>%
    add_proportion_cols() %>%
    select(all_of(group_cols), `Prop Imported`, `Prop Local`, `IRR`)
}

```

```

}

### Correlation plot and table

# This function is for generating the scatter plots.
# Inputs include:
# df      - the county-level data frame from corr_data()
# x_col   - name of the column to plot on the x-axis (as a string)
# y_col   - name of the column to plot on the y-axis (as a string)
# x_lab   - human-readable x-axis label shown on the plot
# y_lab   - human-readable y-axis label shown on the plot
# It returns a ggplot object that can be rendered directly by renderPlot.
build_corr_plot <- function(df, x_col, y_col, x_lab, y_lab) {
  ggplot(df, aes(x = .data[[x_col]], y = .data[[y_col]])) +
    # Each point represents one county. alpha = 0.5 adds transparency
    # so overlapping points (many counties with zero cases) are visible.
    geom_point(color = "#A61C1C", alpha = 0.5, size = 1.8) +
    # geom_smooth adds a linear trend line with a shaded 95% confidence
    # interval. method = "lm" fits a straight line. se = TRUE draws the
    # confidence band. The line color matches the app's red palette.
    geom_smooth(method = "lm", se = TRUE,
      color = "#4A0A0A", fill = "#FADADD", linewidth = 0.9) +
    # comma from the scales package formats axis tick labels with commas
    # (e.g., 1000 becomes "1,000"), matching the style of other plots.
    scale_x_continuous(labels = comma,
      expand = expansion(mult = c(0.02, 0.05))) +
    scale_y_continuous(labels = comma,
      expand = expansion(mult = c(0.02, 0.1))) +
    labs(x = x_lab, y = y_lab) +
    theme_minimal(base_size = 13) +
    theme(
      panel.grid.major = element_line(color = "#eeeeee"),
      panel.grid.minor = element_blank(),
      axis.text.x = element_text(size = 11),
      axis.text.y = element_text(size = 11),
      axis.title.x = element_text(size = 11, margin = margin(t = 8)),
      axis.title.y = element_text(size = 11, margin = margin(r = 8)),
      plot.margin = margin(10, 15, 10, 50)
    )
}

# This function runs cor.test with method = "spearman" on two numeric
# vectors and returns a one-row data frame with three columns:
# Correlation (rho) - the Spearman rank correlation coefficient
# P-value          - probability of observing this rho by chance
# Sample Size (n)  - number of counties used in the test
# exact = FALSE suppresses a warning about ties that occurs when many
# counties share the same count (e.g., zero), which is common here.
build_corr_table <- function(x, y) {
  ct <- cor.test(x, y, method = "spearman", exact = FALSE)
  data.frame(
    `Correlation (rho)` = round(ct$estimate, 3),
    `P-value`          = ifelse(ct$p.value < 0.001,

```

```

        "< 0.001",
        as.character(round(ct$p.value, 3))),
`Sample Size (n)` = length(x),
# This preserves spaces and parentheses in col names
check.names = FALSE
)
}

#####
##### User interface
#####

# The user interface is defined using fluidPage, which creates a responsive
# page layout. The interface is divided into a sidebar panel containing all
# input controls and a main panel containing all output components.
ui <- fluidPage(
  titlePanel("United States Measles Surveillance Dashboard"),

  sidebarLayout(

    # The sidebar panel contains all input controls. Values selected here
    # are passed to the server using the inputId of each control.
    sidebarPanel(
      p("Although measles was declared eliminated in the United States in
        2000 by the Centers for Disease Control and Prevention, recent years
        have shown a concerning resurgence in reported cases. Tracking measles
        cases improves awareness of outbreak trends, patterns of transmission,
        and potential reemergence of disease, which can aid in deploying public
        health initiatives to prevent further spread.",
        br(),
        br()),

      ### Inputs

      # The radioButtons input allows the user to select one map level.
      # The inputId "map_level" is used in the server to read this value.
      # The choices argument defines the display labels and their
      # corresponding values returned to the server.
      radioButtons(
        inputId = "map_level",
        label = "Map level",
        choices = c("State (aggregated)" = "state", "County" = "county"),
        selected = "state"
      ),

      # The selectInput input creates a single-select dropdown for the
      # case type. The outcome_choices vector defined above provides the
      # display labels and corresponding data values.
      selectInput(
        inputId = "outcome",
        label = "Case type",
        choices = outcome_choices,
        selected = "case_lab-confirmed"
      )
    )
  )
)

```

```

),

# The dateRangeInput input creates a start and end date selector.
# The min and max arguments set the earliest and latest selectable
# dates based on the range of dates present in the data.
dateRangeInput(
  inputId = "date_range",
  label   = "Date range",
  min     = date_range[1],
  max     = date_range[2],
  start   = date_range[1],
  end     = date_range[2]
),

# The selectizeInput input creates a multi-select dropdown for states.
# The option "All States" is assigned the sentinel value "ALL" and is
# selected by default. A JavaScript callback defined in the options
# argument enforces mutual exclusivity: selecting "All States" clears
# all other selections, and selecting any real state removes "All States".
# The remove_button plugin adds an X tag to each selected item.
selectizeInput(
  inputId = "state_filter",
  label   = "Filter to state(s)",
  choices = c("All States" = "ALL", sort(unique(raw$state))),
  selected = "ALL",
  multiple = TRUE,
  options = list(
    plugins = list("remove_button"),
    onItemAdd = I("function(value, item) {
      if (value === 'ALL') {
        var current = this.getValue();
        var filtered = current.filter(function(v) { return v === 'ALL'; });
        this.setValue(filtered, true);
      } else {
        var current = this.getValue();
        var filtered = current.filter(function(v) { return v !== 'ALL'; });
        this.setValue(filtered, true);
      }
    })"
  )
),

# The sliderInput below allows the user choose how many states appear
# in the top locations table on the Overview tab and the State Summary
# and County Summary tables. The value is read as input$top_n_states
# in the server. The range is 1 to 10, defaulting to 5.
sliderInput(
  inputId = "top_n_states",
  label   = "Top number of states",
  min     = 1,
  max     = 10,
  value   = 5,
  step    = 1

```



```

),

# The sliderInput below allows the user choose how many counties per
# state appear in the County Summary table. The value is read as
# input$top_n_counties in the server. The range is 1 to 5,
# defaulting to 3.
sliderInput(
  inputId = "top_n_counties",
  label   = "Top number of counties (per state)",
  min     = 1,
  max     = 5,
  value   = 3,
  step    = 1
),

br(),

# The actionButton input creates a button labeled Update. All filtered
# data reactives in the server are gated on this button using bindEvent,
# so outputs only update when the button is clicked.
actionButton(
  inputId = "update",
  label   = "Update",
  width   = "100%",
  class   = "btn-danger"
),

p(br(),
  br(),
  "Data source:",
  a(href="https://github.com/CSSEGISandData/measles_data", "CSSEGISandData"),
  " from the Johns Hopkins University Measles Tracking Team - International
  Vaccine Access Center (IVAC), Bloomberg School of Public Health; Center for
  Systems Science and Engineering (CSSE), Whiting School of Engineering;
  Bloomberg Center for Government Excellence, Johns Hopkins University"),
),

# The main panel contains all output components. Each output function
# below is paired with a corresponding render function in the server.
mainPanel(

  # Four summary boxes are arranged in a single row using fluidRow and
  # column. Each box displays a label and a text output value.
  # These boxes sit above the tabs so they are always visible.
  fluidRow(
    column(3, div(class = "summary-box",
      div(class = "summary-label", "Total Cases"),
      div(class = "summary-value", textOutput("box_total"))
    )),
    column(3, div(class = "summary-box",
      div(class = "summary-label", "Date Range"),
      div(class = "summary-value summary-value-sm", textOutput("box_dates"))
    ))
  )
)

```

```

)),
column(3, div(class = "summary-box",
              div(class = "summary-label", "State Count"),
              div(class = "summary-value", textOutput("box_states")))
)),
column(3, div(class = "summary-box",
              div(class = "summary-label", "County Count"),
              div(class = "summary-value", textOutput("box_counties")))
))
),

```

*# The tags\$style function injects CSS into the page to style the
summary boxes defined above. Styles are applied by class name.*

```

tags$style("
  .summary-box {
    background: #f8f8f8;
    border: 1px solid #e0e0e0;
    border-radius: 8px;
    padding: 14px 16px;
    margin-bottom: 16px;
    text-align: center;
  }
  .summary-label {
    font-size: 12px;
    color: #888;
    text-transform: uppercase;
    letter-spacing: 0.05em;
    margin-bottom: 4px;
  }
  .summary-value {
    font-size: 26px;
    font-weight: 600;
    color: #A61C1C;
  }
  .summary-value-sm {
    font-size: 15px;
    padding-top: 5px;
    color: #333;
  }
")

```

Outputs

*# The tabsetPanel function creates a tab layout. Each tabPanel
defines one tab. Tabs share the sidebar filter controls above.*

```

tabsetPanel(

  # The Overview tab contains the map, case count figures, and top locations table.
  tabPanel(
    "Overview",
    br(),

    # The leafletOutput function declares the map output component.

```

```

# The outputId "map" is used in the server to render and update
# the map. Width is set to 100% and height to 480 pixels.
leafletOutput("map", width = "100%", height = "480px"),
br(),

# The plotOutput function declares the weekly case count plot.
h4("Weekly case counts"),
plotOutput("line_plot", height = "260px"),
br(),

# The plotOutput function declares the cumulative cases by year chart.
h4("Cumulative cases by year"),
plotOutput("cumulative_plot", height = "260px"),
br(),

# The tableOutput function declares the top locations table.
h4("Top locations"),
tableOutput("top_table")
),

# The State Summary tab contains the per-state statistics table.
tabPanel(
  "State Summary",
  br(),

  # A brief description for the table.
  p("Below is a table of summary statistics for the top states
    by total case count, based on the filter settings for case type,
    date range, and selected states. The number of states shown is
    determined by the top number of states slider in the sidebar
    filter settings."),
  br(),
  br(),

  "The descriptive statistics columns (total count, range [minimum,
  maximum], mean, standard deviation, median, interquartile range,
  number of affected counties) are based on the case type selected.
  The statistics are calculated over counties within a given state.",
  br(),
  br(),

  "The proportion and rate columns (proportion of imported cases,
  proportion of local cases, incidence rate ratio) are based on all
  case types. The proportion of imported cases is calculated as the
  number of imported cases divided by the sum of imported and local
  cases. The proportion of local cases is calculated as the number of
  imported cases divided by the sum of imported and local cases. The
  incidence rate ratio (IRR) is calculated as the ratio of unvaccinated
  cases to vaccinated cases."),

  # The dataTableOutput function declares the interactive summary
  # table. The outputId "state_summary_table" is matched to a
  # renderDataTable call in the server.

```

```

    dataTableOutput("state_summary_table")
  ),

  # The County Summary tab contains the per-county statistics table.
  tabPanel(
    "County Summary",
    br(),

    # A brief description for the table.
    p("Below is a table of summary statistics for the top counties
      by total case count within each of the top states, based on the
      filter settings for case type, date range, and selected states.
      The number of states shown is based on the top number of states
      slider, and the number of counties per state shown is based on
      the top number of counties (per state) slider, both of which are in
      the sidebar filter settings.",
      br(),
      br(),

      "The descriptive statistics columns (total count, range [minimum,
      maximum], mean, standard deviation, median, interquartile range,
      number of affected counties) are based on the case type selected.
      The statistics are calculated over reporting days for a given county.",
      br(),
      br(),

      "The proportion and rate columns (proportion of imported cases,
      proportion of local cases, incidence rate ratio) are based on all
      case types. The proportion of imported cases is calculated as the
      number of imported cases divided by the sum of imported and local
      cases. The proportion of local cases is calculated as the number of
      imported cases divided by the sum of imported and local cases. The
      incidence rate ratio (IRR) is calculated as the ratio of unvaccinated
      cases to vaccinated cases."),

    # The dataTableOutput function declares the interactive county
    # summary table. The outputId "county_summary_table" is matched
    # to a renderDataTable call in the server.
    dataTableOutput("county_summary_table")
  ),

  # The Correlation Analysis tab has Spearman correlation figures and tables.
  tabPanel(
    "Correlation Analysis",
    br(),

    # A brief description explaining what the tab shows and how the
    # unit of observation (county-level totals) is derived.
    # Both correlations use the same date and state filters as the
    # other tabs. The Case Type filter not used, as specific case types
    # are specified per figure instead (imported, local, unvaccinated).
    p("Spearman rank correlations between the specified case types,
      computed at the county level using total counts over the selected

```

```

    date range and state filter. Note that case types are fixed by the
    correlation being examined. A scatter plot with a linear trend line
    is shown in each of the figures below with shading as the 95%
    confidence interval."),

# The plotOutput function declares the scatter plot for the
# imported vs. local correlation.
h4("Figure 1: Imported cases vs. local transmission cases"),
plotOutput("corr_plot_imp_local", height = "340px"),
br(),

# The tableOutput function declares the summary statistics table
# (rho, p-value, n) for the imported vs. local correlation.
tableOutput("corr_table_imp_local"),
br(),

# The plotOutput function declares the scatter plot for the
# unvaccinated vs. local correlation.
h4("Figure 2: Unvaccinated cases vs. local transmission cases"),
plotOutput("corr_plot_unvacc_local", height = "340px"),
br(),

# The tableOutput function declares the summary statistics table
# (rho, p-value, n) for the unvaccinated vs. local correlation.
tableOutput("corr_table_unvacc_local")
)
)
)
)
)

#####
##### Server
#####

# The server function contains all reactive logic. It takes input and output
# as arguments. Input values are read using input$inputId and output values
# are assigned using output$outputId.
server <- function(input, output){

#####
##### Reactives
#####

### Helper reactives

# A helper reactive called sel_states is defined to return the vector of
# selected state names with the sentinel value "ALL" removed. This reactive
# is called by multiple other reactives below, so defining it here avoids
# repeating it.
# The reactive returns a character vector of state names, or an empty
# character vector if "All States" is selected.
sel_states <- reactive({

```

```

input$state_filter[input$state_filter != "ALL"]
})

# A helper reactive called plot_subtitle builds the subtitle string
# used on both the weekly case count and cumulative cases plots. It is
# defined once here so the same logic does not need to be repeated inside
# each renderPlot call below.
# Logic:
# - If no specific states are selected, return "All states".
# - If 1-3 states are selected, list their names after a colon.
# - If 4 or more states are selected, show only the count to save space.
plot_subtitle <- reactive({
  sel <- sel_states()
  if (length(sel) == 0) {
    "All states"
  } else if (length(sel) <= 3) {
    paste("Filtered to:", paste(sel, collapse = ", "))
  } else {
    paste0("Filtered to ", length(sel), " states")
  }
})

### Filtered data reactives

# The filtered_raw reactive applies the outcome type, date range, and state
# filters to the raw dataset. It returns a filtered data frame and serves
# as the input to all downstream summary reactives.
# The bindEvent function gates execution on input$update, so this reactive
# only re-runs when the Update button is clicked. The ignoreNULL = FALSE
# argument allows the reactive to run once on startup before any click.
filtered_raw <- reactive({
  # Apply the date and state filters using the helper, then
  # keep only the outcome type the user selected in the dropdown.
  filter_by_date_state(raw,
    date_start = input$date_range[1],
    date_end   = input$date_range[2],
    states      = sel_states()) %>%
    filter(outcome_type == input$outcome)
}) %>% bindEvent(input$update, ignoreNULL = FALSE)

# The filtered_county reactive groups the filtered data by county and
# computes the total case count per county. The output is used for
# the county-level map, the county top locations table, and the
# county count summary box.
filtered_county <- reactive({
  filtered_raw() %>%
    group_by(fips, state, county) %>%
    summarize(cases = sum(value, na.rm = TRUE), .groups = "drop")
}) %>% bindEvent(input$update, ignoreNULL = FALSE)

# The filtered_state reactive groups the filtered data by state and
# computes the total case count per state. The output is used for
# the state-level map, the state top locations table, and the

```

```

# state count summary box.
filtered_state <- reactive({
  filtered_raw() %>%
    group_by(state) %>%
    summarize(cases = sum(value, na.rm = TRUE), .groups = "drop")
}) %>% bindEvent(input$update, ignoreNULL = FALSE)

# The weekly_cases reactive groups the filtered data by week and computes
# the total case count per week. The output is used for the weekly line
# plot. The data is sorted chronologically so the line draws from left
# to right.
weekly_cases <- reactive({
  filtered_raw() %>%
    group_by(week) %>%
    summarize(cases = sum(value, na.rm = TRUE), .groups = "drop") %>%
    arrange(week)
}) %>% bindEvent(input$update, ignoreNULL = FALSE)

# The cumulative_cases reactive groups the filtered data by year and month
# and computes the total case count per year-month combination. Within
# each year, the function cumsum computes a running total of cases so that
# each month's value reflects all cases from January of that year through
# the current month. The year column is converted to a factor so that
# ggplot2 assigns a distinct color to each year rather than treating
# year as a continuous numeric variable.
cumulative_cases <- reactive({
  filtered_raw() %>%
    mutate(
      year = year(date),
      month = month(date)
    ) %>%
    group_by(year, month) %>%
    summarize(cases = sum(value, na.rm = TRUE), .groups = "drop") %>%
    arrange(year, month) %>%
    group_by(year) %>%
    mutate(cumulative = cumsum(cases)) %>%
    ungroup() %>%
    mutate(year = as.factor(year))
}) %>% bindEvent(input$update, ignoreNULL = FALSE)

### Geographical data reactives

# The state_geo reactive downloads state boundary shapefiles using the
# tigris function states and joins the filtered state case counts onto
# the shapefile by matching the Census NAME column to the state column
# in the case data. States with no matching case data are assigned zero.
# If specific states are selected, unselected states are assigned NA so
# they render in the na.color gray defined in make_pal.
state_geo <- reactive({
  sel <- sel_states()
  geo <- states(cb = TRUE, resolution = "20m", year = 2023) %>%
    filter(!STUSPS %in% c("AK", "HI", "PR", "GU", "VI", "MP", "AS")) %>%
    left_join(filtered_state(), by = c("NAME" = "state")) %>%

```

```

    mutate(cases = replace_na(cases, 0))
  if (length(sel) > 0) {
    geo <- geo %>%
      mutate(cases = ifelse(NAME %in% sel, cases, NA_real_))
  }
  geo
})

# The county_geo reactive downloads county boundary shapefiles using the
# tigris function counties. If specific states are selected, only counties
# belonging to those states are downloaded by passing the corresponding
# two-digit FIPS state codes to the state argument of counties. The
# filtered county case counts are then joined onto the shapefile using
# the five-digit FIPS code. Counties with no matching case data are
# assigned zero.
county_geo <- reactive({
  sel <- sel_states()
  if (length(sel) == 0) {
    geo <- counties(cb = TRUE, resolution = "20m", year = 2023) %>%
      filter(!STATEFP %in% c("02", "15", "72", "66", "78", "60", "69"))
  } else {
    fp <- fips_codes %>%
      filter(state_name %in% sel) %>%
      pull(state_code) %>%
      unique()
    geo <- counties(state = fp, cb = TRUE, year = 2023)
  }
  geo %>%
    left_join(filtered_county(), by = c("GEOID" = "fips")) %>%
    mutate(
      cases = replace_na(cases, 0),
      # STATE_NAME from tigris is always populated; state from the join
      # is NA for any county with zero cases. coalesce picks the first non-NA.
      state = coalesce(state, STATE_NAME)
    )
})

### State summary table reactive

state_summary_data <- reactive({

  # Part A: Descriptive statistics for the selected outcome type.
  # Summarize at county level first (one row per county), then roll up
  # to state level. "Mean" addresses the question: on average, how many
  # cases did an affected county in this state report?
  county_level <- filtered_raw() %>%
    group_by(state, fips, county) %>%
    summarize(cases = sum(value, na.rm = TRUE), .groups = "drop")

  stats_a <- county_level %>%
    group_by(state) %>%
    summarize(
      `Total Count` = sum(cases, na.rm = TRUE),

```



```

    .min          = min(cases, na.rm = TRUE),
    .max          = max(cases, na.rm = TRUE),
    Mean          = round(mean(cases, na.rm = TRUE), 2),
    `Std Dev`    = round(sd(cases, na.rm = TRUE), 2),
    Median        = round(median(cases, na.rm = TRUE), 2),
    IQR           = round(IQR(cases, na.rm = TRUE), 2),
    `Affected Counties` = sum(cases > 0, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  mutate(Range = paste0("[", .min, ", ", .max, "]")) %>%
  select(-c(.min, .max))

# Part B: Proportion and rate columns (all outcome types).
# build_outcome_wide applies only date and state filters, pivots outcome
# types wide, and computes Prop Imported, Prop Local, and IRR.
stats_b <- build_outcome_wide(raw,
                              date_start = input$date_range[1],
                              date_end   = input$date_range[2],
                              states     = sel_states(),
                              group_cols = "state")

# Join and keep top N states by total count
stats_a %>%
  left_join(stats_b, by = "state") %>%
  arrange(desc(`Total Count`)) %>%
  slice_head(n = input$top_n_states) %>%
  select(State = state, `Total Count`, Range, Mean, `Std Dev`,
         Median, IQR, `Prop Imported`, `Prop Local`, `IRR`,
         `Affected Counties`)

}) %>% bindEvent(input$update, ignoreNULL = FALSE)

### County summary table reactive

county_summary_data <- reactive({

  # Part A: Descriptive statistics for the selected outcome type.
  # Each row in filtered_raw() is one county on one date. Statistics are
  # computed across reporting periods within each county. "Mean" addresses
  # the question: on an average reporting day, how many cases did this
  # county report?
  stats_a <- filtered_raw() %>%
    group_by(state, county) %>%
    summarize(
      `Total Count` = sum(value, na.rm = TRUE),
      .min          = min(value, na.rm = TRUE),
      .max          = max(value, na.rm = TRUE),
      Mean          = round(mean(value, na.rm = TRUE), 2),
      `Std Dev`    = round(sd(value, na.rm = TRUE), 2),
      Median        = round(median(value, na.rm = TRUE), 2),
      IQR           = round(IQR(value, na.rm = TRUE), 2),
      `Affected Counties` = as.integer(sum(value, na.rm = TRUE) > 0),
      .groups = "drop"
    )

```

```

) %>%
mutate(Range = paste0("[", .min, ", ", .max, "]")) %>%
select(-c(.min, .max))

# Identify top N states and top N counties per state
top_states <- stats_a %>%
  group_by(state) %>%
  summarize(state_total = sum(`Total Count`, na.rm = TRUE),
    .groups = "drop") %>%
  arrange(desc(state_total)) %>%
  slice_head(n = input$top_n_states) %>%
  pull(state)

stats_a <- stats_a %>%
  filter(state %in% top_states) %>%
  group_by(state) %>%
  slice_max(`Total Count`, n = input$top_n_counties, with_ties = FALSE) %>%
  ungroup()

# Part B: Proportion and rate columns (all outcome types).
# Same helper as the state reactive, but grouped by state and county.
stats_b <- build_outcome_wide(raw,
  date_start = input$date_range[1],
  date_end   = input$date_range[2],
  states     = sel_states(),
  group_cols = c("state", "county"))

# Join and enforce top-state ordering
stats_a %>%
  left_join(stats_b, by = c("state", "county")) %>%
  mutate(state = factor(state, levels = top_states)) %>%
  arrange(state, desc(`Total Count`)) %>%
  mutate(state = as.character(state)) %>%
  select(State = state, County = county, `Total Count`, Range,
    Mean, `Std Dev`, Median, IQR, `Prop Imported`, `Prop Local`,
    `IRR`, `Affected Counties`)

}) %>% bindEvent(input$update, ignoreNULL = FALSE)

### Correlation data reactive

# The corr_data reactive prepares the county-level data used by both
# Spearman correlation figures on the Correlation Analysis tab.
# It is gated on input$update like all other filtered reactives so
# all tabs update at the same time when the user clicks Update.

# The unit of observation is one county. For each county we compute
# the total case count for three specific outcome types over the
# selected date range:
#   case_imported - imported cases
#   case_local    - local transmission cases
#   case_unvaccinated - cases in unvaccinated individuals

```

```

# We do not filter by input$outcome here because the correlations
# always compare fixed pairs of outcome types, regardless of what the
# user has selected in the Case Type dropdown.
corr_data <- reactive({

  # Start from raw data and apply only the date and state filters using the
  # helper function. We do not filter by outcome type here because the
  # correlations always compare fixed pairs of outcome types, regardless
  # of what the user selected in the Case Type dropdown.
  d <- filter_by_date_state(raw,
    date_start = input$date_range[1],
    date_end   = input$date_range[2],
    states     = sel_states()) %>%

  filter(
    # Keep only the three outcome types needed for both correlations.
    # This reduces the data size before the group-by step below.
    outcome_type %in% c("case_imported", "case_local",
      "case_unvaccinated")
  )

  # Aggregate to county-level totals, then pivot so each outcome type
  # becomes its own column. This gives one row per county with columns
  # case_imported, case_local, and case_unvaccinated.
  d %>%
  group_by(state, county, outcome_type) %>%
  summarize(total = sum(value, na.rm = TRUE), .groups = "drop") %>%
  pivot_wider(names_from = outcome_type, values_from = total,
    values_fill = 0) %>%
  # Use ensure_outcome_cols() to guarantee all three required
  # columns are present even if a narrow filter produces no rows for
  # one of the outcome types.
  ensure_outcome_cols(c("case_imported", "case_local",
    "case_unvaccinated"))

}) %>% bindEvent(input$update, ignoreNULL = FALSE)

#####
##### Upper panel outputs: summary boxes
#####

# The total cases output sums all values in the value column of the
# filtered raw data. The comma function from the scales package formats
# the result with comma separators.
output$box_total <- renderText({
  comma(sum(filtered_raw()$value, na.rm = TRUE))
})

# The date range output formats the selected start and end dates as
# month, day, and year strings separated by an em dash.
output$box_dates <- renderText({
  paste0(format(input$date_range[1], "%b %d, %Y"),
    " - ",
    format(input$date_range[2], "%b %d, %Y"))
})

```

```

}) %>% bindEvent(input$update, ignoreNULL = FALSE)

# The state count output returns the number of distinct states present
# in the filtered state summary data.
output$box_states <- renderText({
  n_distinct(filtered_state())$state
})

# The county count output returns the number of counties in the filtered
# county summary data that have at least one reported case.
output$box_counties <- renderText({
  filtered_county() %>% filter(cases > 0) %>% nrow()
})

#####
##### Overview tab outputs
#####

### Interactive map

# The renderLeaflet function creates the base map once on startup with a
# light CartoDB tile layer centered on the contiguous United States.
# Subsequent updates to the map polygons are handled by leafletProxy,
# which modifies the existing map in place without redrawing it.
output$map <- renderLeaflet({
  leaflet() %>%
    addProviderTiles(providers$CartoDB.PositronNoLabels) %>%
    setView(lng = -96, lat = 38, zoom = 4)
})

# The observe block updates the map polygons and legend whenever the
# Update button is clicked or the map level radio button is changed.
# The bindEvent function attends to both input$update and input$map_level,
# so that switching between state and county view takes effect immediately
# without requiring a button click.
observe({
  if (input$map_level == "state") {

    # For the state map, the color palette is built from non-NA case values
    # only. Tooltip labels are constructed as HTML strings and converted
    # using htmltools::HTML so that the bold tags render correctly.
    geo <- state_geo()
    pal <- make_pal(geo$cases[!is.na(geo$cases)])
    labs <- sprintf("<strong>%s</strong><br/>%s cases",
                    geo$NAME,
                    ifelse(is.na(geo$cases), "Not selected",
                           comma(geo$cases))) %>%
      lapply(htmltools::HTML)

    leafletProxy("map") %>%
      clearShapes() %>%
      clearControls() %>%
      addPolygons(

```

```

    data      = geo,
    fillColor = ~pal(cases),
    fillOpacity = 0.8,
    color      = "#AAAAAA",
    weight     = 1.5,
    highlight  = highlightOptions(weight = 2.5, color = "#ff6b35",
                                   bringToFront = TRUE),

    label      = labs,
    labelOptions = labelOptions(style = list("font-size" = "13px"))
  ) %>%
  addLegend(
    pal      = pal,
    values   = geo$cases[!is.na(geo$cases)],
    title    = "Total cases",
    position = "bottomright",
    labFormat = labelFormat(big.mark = ",")
  )
} else {

  # For the county map, the border weight is set to 0.4, which is thinner
  # than the state border weight of 1.5, to prevent the county grid from
  # appearing visually dense if zoomed out to view entire country.
  geo <- county_geo()
  pal <- make_pal(geo$cases)
  labs <- sprintf("<strong>%s, %s</strong><br/>%s cases",
                  geo$NAME, geo$state, comma(geo$cases)) %>%
    lapply(htmltools::HTML)

  leafletProxy("map") %>%
    clearShapes() %>%
    clearControls() %>%
    addPolygons(
      data      = geo,
      fillColor = ~pal(cases),
      fillOpacity = 0.8,
      color      = "#AAAAAA",
      weight     = 0.4,
      highlight  = highlightOptions(weight = 1.5, color = "#ff6b35",
                                   bringToFront = TRUE),

      label      = labs,
      labelOptions = labelOptions(style = list("font-size" = "12px"))
    ) %>%
    addLegend(
      pal      = pal,
      values   = geo$cases,
      title    = "Total cases",
      position = "bottomright",
      labFormat = labelFormat(big.mark = ",")
    )
  }
} %>% bindEvent(input$update, input$map_level, ignoreNULL = FALSE)

```

Weekly case count plot

```
# The renderPlot function creates the weekly case count line plot using
# ggplot2. The x-axis displays weeks with tick marks placed at the first
# week of each calendar month, labeled in the format "Jan '25". The
# y-axis displays the case count formatted with comma separators.
# A shaded area is drawn beneath the line using geom_area.
output$line_plot <- renderPlot({
  wk <- weekly_cases()

  # The x-axis tick positions are set to the earliest week date falling
  # within each calendar month. The slice_min function retains only one
  # row per month group, corresponding to the earliest week in that month.
  month_breaks <- wk %>%
    mutate(month = floor_date(week, "month")) %>%
    group_by(month) %>%
    slice_min(week, n = 1) %>%
    pull(week)

  ggplot(wk, aes(x = week, y = cases)) +
    geom_area(fill = "#FADADD", alpha = 0.5) +
    geom_line(color = "#A61C1C", linewidth = 0.9) +
    geom_point(color = "#A61C1C", size = 1.8) +
    scale_x_date(
      breaks      = month_breaks,
      labels      = function(x) format(x, "%b '%y"),
      minor_breaks = NULL
    ) +
    scale_y_continuous(
      labels = comma,
      expand = expansion(mult = c(0, 0.1))
    ) +
    labs(x = NULL, y = "Cases", subtitle = plot_subtitle()) +
    theme_minimal(base_size = 13) +
    theme(
      panel.grid.major.x = element_blank(),
      panel.grid.minor   = element_blank(),
      panel.grid.major.y = element_line(color = "#eeeeee"),
      plot.subtitle      = element_text(color = "#888888", size = 11),
      axis.text.x         = element_text(angle = 45, hjust = 1, size = 11),
      axis.text.y         = element_text(size = 11),
      plot.margin         = margin(8, 16, 8, 8)
    )
}, res = 110)
```

Cumulative cases by year chart

```
# The renderPlot function creates the cumulative cases by year chart using
# ggplot2. Each calendar year is drawn as a separate line. The x-axis
# displays month abbreviations from January through December. The y-axis
# displays the running cumulative case count from the start of each year.
# The year_colors vector defined at startup maps each year to a red shade.
output$cumulative_plot <- renderPlot({
```

```

cum <- cumulative_cases()

# The x-axis tick marks are set to integers 1 through 12 corresponding
# to each calendar month. The built-in R constant month.abb provides
# the three-letter abbreviations Jan through Dec as tick labels.
ggplot(cum, aes(x = month, y = cumulative, color = year, group = year)) +
  geom_line(linewidth = 0.9) +
  geom_point(size = 1.8) +
  scale_x_continuous(
    breaks = 1:12,
    labels = month.abb,
    limits = c(1, 12)
  ) +
  scale_y_continuous(
    labels = comma,
    expand = expansion(mult = c(0, 0.1))
  ) +
  scale_color_manual(values = year_colors) +
  labs(
    x = NULL,
    y = "Cumulative cases",
    color = "Year",
    subtitle = plot_subtitle()
  ) +
  theme_minimal(base_size = 13) +
  theme(
    panel.grid.major.x = element_blank(),
    panel.grid.minor = element_blank(),
    panel.grid.major.y = element_line(color = "#eeeeee"),
    plot.subtitle = element_text(color = "#888888", size = 11),
    axis.text.x = element_text(size = 11),
    axis.text.y = element_text(size = 11),
    legend.position = "right",
    plot.margin = margin(8, 16, 8, 8)
  )
}, res = 110)

```

Top locations table

```

# The renderTable function creates the top locations table. The table
# displays the top N locations with the highest case counts under the
# current filter settings, where N is controlled by the Top States slider.
# When the map level is set to state, one row per state is returned.
# When set to county, one row per county is returned with a combined
# location label in the format "County, State".
# The digits argument suppresses decimal places and the format.args
# argument adds comma separators to case count values.
output$top_table <- renderTable({
  if (input$map_level == "state") {
    filtered_state() %>%
      arrange(desc(cases)) %>%
      # Use input$top_n_states so this table respects the slider.
      slice_head(n = input$top_n_states) %>%

```

```

    rename(State = state, `Total cases` = cases)
  } else {
    filtered_county() %>%
      arrange(desc(cases)) %>%
      # Use input$top_n_states so this table respects the slider.
      slice_head(n = input$top_n_states) %>%
      mutate(Location = paste0(county, " County, ", state)) %>%
      select(Location, `Total cases` = cases)
  }
}, digits = 0, format.args = list(big.mark = ","))

#####
##### State summary tab table output
#####

# The renderDataTable function creates the interactive per-state statistics
# table on the State Summary tab. DT::renderDataTable is called explicitly
# to use the DT package version (which supports column formatting options)
# rather than the base Shiny version.
output$state_summary_table <- DT::renderDataTable({

  df <- state_summary_data()

  # The datatable function from DT creates an interactive HTML table with
  # built-in search, sort, and pagination controls. The options argument
  # suppresses the row count selector (pageLength = 10 shows all rows
  # since we cap at 10 states) and centers the table on the page.
  DT::datatable(
    df,
    rownames = FALSE,
    options = list(
      pageLength = 10,      # Show all rows without pagination controls
      dom               = "t", # "t" means show only the table, no search bar
      scrollX           = TRUE  # Allow horizontal scroll for wide tables
    )
  ) %>%
  # Format Total Count and Affected Counties as integers with commas.
  DT::formatCurrency(
    columns = c("Total Count", "Affected Counties"),
    currency = "",
    digits = 0,
    mark = ",",
  ) %>%
  # Format Mean, Std Dev, Median, and IQR to two decimal places.
  DT::formatRound(
    columns = c("Mean", "Std Dev", "Median", "IQR"),
    digits = 2
  ) %>%
  # Format proportion and IRR columns to three decimal places.
  # NA values (e.g., division by zero) will display as a dash.
  DT::formatRound(
    columns = c("Prop Imported", "Prop Local", "IRR"),
    digits = 3
  )
})

```



```

    )
  })

#####
##### County summary tab table output
#####

# The renderDataTable function creates the interactive per-county statistics
# table on the County Summary tab. DT::renderDataTable is called explicitly
# to use the DT package version (which supports column formatting options)
# rather than the base Shiny version.
output$county_summary_table <- DT::renderDataTable({

  df <- county_summary_data()

  # The datatable function creates an interactive HTML table with built-in
  # sort and scroll controls. pageLength is set to 50 (the maximum possible
  # rows: 10 states x 5 counties) so all rows are always visible without
  # pagination, regardless of the current slider settings.
  # dom = "t" hides the search bar so the table stays compact.
  # scrollX allows horizontal scrolling on narrow screens.
  DT::datatable(
    df,
    rownames = FALSE,
    options = list(
      pageLength = 50,      # Show all rows (up to 10 states x 5 counties)
      dom          = "t",   # "t" means show only the table, no search bar
      scrollX       = TRUE   # Allow horizontal scroll for wide tables
    )
  ) %>%
  # Format Total Count and Affected Counties as integers with commas.
  DT::formatCurrency(
    columns = c("Total Count", "Affected Counties"),
    currency = "",
    digits   = 0,
    mark     = ",",
  ) %>%
  # Format Mean, Std Dev, Median, and IQR to two decimal places.
  DT::formatRound(
    columns = c("Mean", "Std Dev", "Median", "IQR"),
    digits  = 2
  ) %>%
  # Format proportion and IRR columns to three decimal places.
  # NA values (e.g., division by zero) will display as a dash.
  DT::formatRound(
    columns = c("Prop Imported", "Prop Local", "IRR"),
    digits  = 3
  )
})

#####
##### Correlation analysis tab outputs: plots and tables
#####

```

```

# Figure 1: Imported cases vs. Local transmission.
# The renderPlot function builds the scatter plot for Figure 1.
# The entire body is wrapped in tryCatch so that if the filtered data
# lacks sufficient variance (e.g., all-zero columns after a narrow filter),
# the plot area shows a readable message instead of a red error screen.
output$corr_plot_imp_local <- renderPlot({
  tryCatch({
    df <- corr_data()
    build_corr_plot(
      df = df,
      x_col = "case_imported",
      y_col = "case_local",
      x_lab = "Imported cases \n(county total)",
      y_lab = "Local transmission cases \n(county total)"
    )
  }, error = function(e) {
    # Display a plain-text message if the plot cannot be generated.
    # This happens when there is not enough data variation after filtering.
    ggplot() +
      annotate("text", x = 0.5, y = 0.5,
        label = paste("Insufficient data to generate this figure.",
          "\nTry expanding the date range or state filter."),
        size = 5, color = "#888888", hjust = 0.5, vjust = 0.5) +
      theme_void()
  })
}, res = 110)

# The renderTable function builds the summary statistics table for
# Figure 1. digits = 3 limits decimal places for numeric columns.
output$corr_table_imp_local <- renderTable({
  df <- corr_data()
  build_corr_table(df$case_imported, df$case_local)
}, digits = 3)

# Figure 2: Unvaccinated cases vs. Local transmission.
# The renderPlot function builds the scatter plot for Figure 2.
# The entire body is wrapped in tryCatch for the same reason as Figure 1.
output$corr_plot_unvacc_local <- renderPlot({
  tryCatch({
    df <- corr_data()
    build_corr_plot(
      df = df,
      x_col = "case_unvaccinated",
      y_col = "case_local",
      x_lab = "Unvaccinated cases \n(county total)",
      y_lab = "Local transmission cases \n(county total)"
    )
  }, error = function(e) {
    # Display a plain-text message if the plot cannot be generated.
    ggplot() +
      annotate("text", x = 0.5, y = 0.5,
        label = paste("Insufficient data to generate this figure.",
          "\nTry expanding the date range or state filter."),

```

```

        size = 5, color = "#888888", hjust = 0.5, vjust = 0.5) +
    theme_void()
  })
}, res = 110)

# The renderTable function builds the summary statistics table for
# Figure 2.
output$corr_table_unvacc_local <- renderTable({
  df <- corr_data()
  build_corr_table(df$case_unvaccinated, df$case_local)
}, digits = 3)
}

#####
##### Launch the shiny app
#####

# The shinyApp function connects the ui and server objects defined
# above and launches the application.
shinyApp(ui = ui, server = server)

```